📚 Instrucciones Completas - Aplicación Fullstack React + Express + MongoDB

Esta guía te llevará paso a paso en la creación de una aplicación fullstack desde cero, comenzando por el backend con Express, MongoDB y TypeScript, seguido del frontend con React, TypeScript y Vite.

📋 Tabla de Contenidos

1. Configuración Inicial del Backend
2. Modelo de Usuario (User.ts)
3. Rutas de Usuario (user.routes.ts)
4. Servidor de Aplicación (app.ts)
5. Punto de Entrada y Conexión a MongoDB (index.ts)
6. Configuración del package.json del Backend
7. Configuración Inicial del Frontend
8. Servicio de API (api.ts)
9. Componente Principal de React (App.tsx)
10. Sistema de Validaciones
11. Elemento de Carga (Loading)
12. Sistema de Alertas de Mensajes

PARTE 1: BACKEND (Express + MongoDB + TypeScript)

---

1. Configuración Inicial del Backend

Crear el proyecto desde cero

```
mkdir proyect-fullstack
cd proyect-fullstack
npm init -y
```

Instalar dependencias principales

```
npm install express mongoose cors dotenv axios
```

Explicación de las dependencias:

- **express**: Framework web para crear el servidor HTTP
- **mongoose**: ODM (Object Document Mapper) para MongoDB
- **cors**: Middleware para habilitar CORS y permitir peticiones desde el frontend

- **dotenv**: Para manejar variables de entorno
- **axios**: Cliente HTTP (usado tanto en frontend como backend)

Instalar dependencias de desarrollo

```
npm install -D typescript @types/express @types/node @types/cors ts-node-dev
```

Explicación de las dependencias de desarrollo:

- **typescript**: Compilador de TypeScript
- **@types/express**: Tipos de TypeScript para Express
- **@types/node**: Tipos de TypeScript para Node.js
- **@types/cors**: Tipos de TypeScript para CORS
- **ts-node-dev**: Ejecuta TypeScript directamente con recarga automática

Inicializar TypeScript

```
npx tsc --init
```

Esto crea un archivo `tsconfig.json` con la configuración de TypeScript.

Estructura de carpetas del backend

Crea la siguiente estructura:

```
proyect-fullstack/
├── src/
│   ├── models/
│   │   └── User.ts
│   ├── routes/
│   │   └── user.routes.ts
│   ├── app.ts
│   └── index.ts
├── package.json
└── tsconfig.json
```

---

2. Modelo de Usuario (User.ts)

Ubicación: `src/models/User.ts`

Código Completo del Archivo:

```
import { model, Schema } from "mongoose";
```

```
export interface IUser {
  name: string;
  email: string;
}

const UserSchema = new Schema<IUser>(
  {
    name: { type: String, required: true },
    email: { type: String, required: true },
  },
  { timestamps: true },
);

export default model<IUser>("User", UserSchema);
```

Explicación Detallada:

1. Importaciones

```
import { model, Schema } from "mongoose";
```

- **Schema**: Clase de Mongoose para definir la estructura de un documento
- **model**: Función para crear un modelo a partir del esquema

2. Interfaz TypeScript

```
export interface IUser {
  name: string;
  email: string;
}
```

- Define la estructura de tipos de un usuario en TypeScript
- **name**: Campo de texto para el nombre del usuario
- **email**: Campo de texto para el email del usuario
- Se exporta para poder usarla en otros archivos

3. Esquema de Mongoose

```
const UserSchema = new Schema<IUser>(
  {
    name: { type: String, required: true },
    email: { type: String, required: true },
  },
  { timestamps: true },
);
```

- **new Schema**: Crea un nuevo esquema tipado con la interfaz IUser
- **name**: Campo de tipo String y obligatorio (required: true)
- **email**: Campo de tipo String y obligatorio
- **timestamps: true**: Añade automáticamente los campos `createdAt` y `updatedAt` a cada documento

4. Exportación del Modelo

```
export default model<IUser>("User", UserSchema);
```

- **model**: Crea un modelo de Mongoose tipado
- **'User'**: Nombre del modelo (Mongoose creará la colección 'users' en MongoDB)
- **UserSchema**: El esquema que define la estructura del documento
- **export default**: Se exporta como exportación por defecto para importarlo fácilmente

---

3. Rutas de Usuario (user.routes.ts)

Ubicación: `src/routes/user.routes.ts`

Código Completo del Archivo:

```
import { Router } from "express";
import User from "../models/User";

const router = Router();

//Obtener Usuarios
router.get("/", async (_req, res) => {
  const users = await User.find();
  res.json(users);
});

router.post("/", async (req, res) => {
  const newUser = new User(req.body);
  await newUser.save();
  res.status(201).json({ message: "User Created", user: newUser });
});

export default router;
```

Explicación Detallada:

1. Importaciones

```
import { Router } from "express";
import User from "../models/User";
```

- **Router**: Clase de Express para crear rutas modulares
- **User**: El modelo de usuario que creamos anteriormente

2. Crear instancia del Router

```
const router = Router();
```

- Crea una instancia del router de Express
- Esto nos permite definir rutas que luego se montarán en el servidor principal

3. Ruta GET - Obtener todos los usuarios

```
router.get("/", async (_req, res) => {
  const users = await User.find();
  res.json(users);
});
```

- **router.get('/')**: Define una ruta GET en la raíz del router
- **async**: La función es asíncrona porque trabajamos con la base de datos
- **_req**: Parámetro request (con _ porque no se usa)
- **res**: Parámetro response para enviar la respuesta
- **User.find()**: Método de Mongoose que busca todos los documentos en la colección
- **await**: Espera a que la promesa de find() se resuelva
- **res.json(users)**: Envía los usuarios en formato JSON con código 200 (OK)

4. Ruta POST - Crear un nuevo usuario

```
router.post("/", async (req, res) => {
  const newUser = new User(req.body);
  await newUser.save();
  res.status(201).json({ message: "User Created", user: newUser });
});
```

- **router.post('/')**: Define una ruta POST en la raíz del router
- **req.body**: Contiene los datos enviados en el cuerpo de la petición
- **new User(req.body)**: Crea una nueva instancia del modelo User con los datos recibidos
- **await newUser.save()**: Guarda el usuario en MongoDB
- **res.status(201)**: Establece el código de estado 201 (Created)
- **res.json(...)**: Envía una respuesta JSON con un mensaje de éxito y el usuario creado

5. Exportación del Router

```
export default router;
```

- Exporta el router para poder importarlo en `app.ts`

---

4. Servidor de Aplicación (app.ts)

Ubicación: `src/app.ts`

Código Completo del Archivo:

```
import express from "express";
import cors from "cors";
import userRoutes from "./routes/user.routes";

const app = express();
app.use(express.json());
app.use(cors());

app.use("/api/users", userRoutes);

export default app;
```

Explicación Detallada:

1. Importaciones

```
import express from "express";
import cors from "cors";
import userRoutes from "./routes/user.routes";
```

- **express**: Framework web para Node.js
- **cors**: Middleware para habilitar CORS
- **userRoutes**: Las rutas de usuario que definimos anteriormente

2. Crear la aplicación Express

```
const app = express();
```

- Crea una instancia de la aplicación Express
- Esta instancia será nuestro servidor

3. Middleware para parsear JSON

```
app.use(express.json());
```

- **express.json()**: Middleware que parsea el cuerpo de las peticiones JSON
- Sin esto, no podríamos acceder a `req.body` en las rutas POST
- Debe estar antes de definir las rutas

4. Middleware de CORS

```
app.use(cors());
```

- **cors()**: Habilita CORS (Cross-Origin Resource Sharing)
- Permite que el frontend (en puerto 5173) haga peticiones al backend (en puerto 4000)
- Sin esto, el navegador bloquearía las peticiones por política de mismo origen

5. Montar las rutas de usuario

```
app.use("/api/users", userRoutes);
```

- **app.use()**: Monta un middleware en una ruta específica
- **'/api/users'**: Prefijo de la ruta (todas las rutas de userRoutes empezarán con /api/users)
- **userRoutes**: El router que contiene las rutas GET y POST
- Resultado:
  - GET http://localhost:4000/api/users → Obtener usuarios
  - POST http://localhost:4000/api/users → Crear usuario

6. Exportación de la aplicación

```
export default app;
```

- Exporta la aplicación para ser usada en `index.ts`
- Separa la configuración del servidor de su inicialización

---

5. Punto de Entrada y Conexión a MongoDB (index.ts)

Ubicación: `src/index.ts`

Código Completo del Archivo:

```
import mongoose from "mongoose";
import app from "./app";

const PORT = 4000;

mongoose
  .connect("mongodb://127.0.0.1:27017/react_express_db")
  .then(() => {
    console.log("Connected to MongoDB");
    app.listen(PORT, () => {
      console.log(`Server is running on port ${PORT}`);
    });
  })
  .catch((error) => {
    console.error("Error connecting to MongoDB:", error);
  });
```

Explicación Detallada:

1. Importaciones

```
import mongoose from "mongoose";
import app from "./app";
```

- **mongoose**: Librería ODM para conectarse a MongoDB
- **app**: La aplicación Express que configuramos en app.ts

2. Definir el puerto

```
const PORT = 4000;
```

- Define el puerto en el que el servidor escuchará las peticiones
- 4000 es un puerto común para desarrollo

3. Conectar a MongoDB

```
mongoose.connect("mongodb://127.0.0.1:27017/react_express_db");
```

- **mongoose.connect()**: Método para conectarse a MongoDB
- **mongodb://127.0.0.1:27017**: URL de conexión a MongoDB local
  - **127.0.0.1**: Dirección IP local (localhost)
  - **27017**: Puerto por defecto de MongoDB
- **react_express_db**: Nombre de la base de datos (se crea automáticamente si no existe)

- Retorna una promesa

4. Si la conexión es exitosa

```
.then(() => {
    console.log("Connected to MongoDB");
    app.listen(PORT, () => {
    console.log(`Server is running on port ${PORT}`);
});
})
```

- **.then()**: Se ejecuta si la promesa de conexión se resuelve exitosamente
- **console.log("Connected to MongoDB")**: Mensaje de confirmación
- **app.listen(PORT, callback)**: Inicia el servidor Express
    - Solo inicia el servidor después de conectarse a MongoDB
    - Esto asegura que la base de datos esté disponible antes de recibir peticiones

5. Si la conexión falla

```
.catch((error) => {
    console.error("Error connecting to MongoDB:", error);
});
```

- **.catch()**: Se ejecuta si la promesa de conexión es rechazada
- Muestra el error en la consola
- El servidor no se inicia si hay un error de conexión

---

6. Configuración del package.json del Backend

Ubicación: `package.json` (raíz del proyecto)

Código Completo del Archivo:

```
{
  "name": "proyect-fullstack",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "dev": "ts-node-dev src/index.ts"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
```

```
    "axios": "^1.13.2",
    "cors": "^2.8.6",
    "dotenv": "^17.2.3",
    "express": "^5.2.1",
    "mongoose": "^9.1.5"
  },
  "devDependencies": {
    "@types/cors": "^2.8.19",
    "@types/express": "^5.0.6",
    "@types/node": "^25.0.10",
    "ts-node-dev": "^2.0.0",
    "typescript": "^5.9.3"
  }
}
```

Explicación Detallada:

1. Script de desarrollo

```
"scripts": {
    "dev": "ts-node-dev src/index.ts"
}
```

- **ts-node-dev**: Ejecuta archivos TypeScript directamente sin compilar
- **src/index.ts**: Punto de entrada del servidor
- **Ventajas**:
  - Recarga automática cuando detecta cambios en los archivos
  - No necesita compilar a JavaScript manualmente
  - Ideal para desarrollo

Para ejecutar el backend:

```
npm run dev
```

PARTE 2: FRONTEND (React + TypeScript + Vite)

---

7. Configuración Inicial del Frontend

Crear el proyecto React con Vite

Desde la raíz del proyecto:

```
npm create vite@latest frontend -- --template react-ts
cd frontend
```

```
npm install
```

Explicación:

- **vite@latest**: Usa la última versión de Vite
- **frontend**: Nombre de la carpeta del proyecto
- **--template react-ts**: Template de React con TypeScript

Instalar dependencias adicionales

```
npm install bootstrap axios
```

Explicación:

- **bootstrap**: Framework CSS para estilos
- **axios**: Cliente HTTP para hacer peticiones al backend

Importar Bootstrap

Edita `frontend/src/main.tsx` y añade al principio:

```
import "bootstrap/dist/css/bootstrap.min.css";
```

Estructura de carpetas del frontend

```
frontend/
├── src/
│   ├── services/
│   │   └── api.ts
│   ├── App.tsx
│   ├── main.tsx
│   └── index.css
├── package.json
└── vite.config.ts
```

---

8. Servicio de API (api.ts)

Ubicación: `frontend/src/services/api.ts`

Código Completo del Archivo:

```
const API_URL = "http://localhost:4000/api/users";
```

```
import axios from "axios";

export interface User {
  _id?: string;
  name: string;
  email: string;
}

// export const getUsers = async (): Promise<User[]> => {
//     const res = await fetch(API_URL);
//     return res.json();
// }

// export const createUser = async ( user:User) => {
//     await fetch(API_URL, {
//         method: 'POST',
//         headers: {'Content-Type' : 'application/json'},
//         body: JSON.stringify(user)
//     });
// }

export const getUsers = async (): Promise<User[]> => {
  const res = await axios.get<User[]>(API_URL);
  return res.data;
};

export const createUser = async (user: User) => {
  await axios.post(API_URL, user);
};
```

Explicación Detallada:

1. URL de la API

```
const API_URL = "http://localhost:4000/api/users";
```

- Define la URL base del backend
- **localhost:4000**: Donde corre el servidor Express
- **/api/users**: Ruta que definimos en app.ts

2. Importación de Axios

```
import axios from "axios";
```

- Axios es una librería HTTP más simple y potente que fetch

3. Interfaz de Usuario

```
export interface User {
  _id?: string;
  name: string;
  email: string;
}
```

- **_id?**: ID opcional (con ?) porque MongoDB lo genera automáticamente
- **name**: Nombre del usuario (obligatorio)
- **email**: Email del usuario (obligatorio)
- Se exporta para usarla en otros componentes

4. Función para obtener usuarios (GET)

Versión con Axios (implementada):

```
export const getUsers = async (): Promise<User[]> => {
  const res = await axios.get<User[]>(API_URL);
  return res.data;
};
```

- **async**: Función asíncrona
- **Promise<User[]>**: Retorna una promesa que resuelve un array de usuarios
- **axios.get<User[]>**: Hace una petición GET y espera un array de usuarios
- **res.data**: Axios ya parsea el JSON automáticamente
- Mucho más simple que fetch

Versión con Fetch (comentada):

```
// export const getUsers = async (): Promise<User[]> => {
//     const res = await fetch(API_URL);
//     return res.json();
// }
```

- **fetch(API_URL)**: Hace la petición GET
- **res.json()**: Parsea la respuesta JSON manualmente
- Requiere más pasos que axios

5. Función para crear usuarios (POST)

Versión con Axios (implementada):

```
export const createUser = async (user: User) => {
  await axios.post(API_URL, user);
};
```

- **user:User**: Recibe un objeto de tipo User
- **axios.post(URL, data)**: Hace una petición POST
- Axios envía automáticamente:
  - El Content-Type como 'application/json'
  - El objeto convertido a JSON
- Muy conciso y limpio

Versión con Fetch (comentada):

```
// export const createUser = async ( user:User) => {
//     await fetch(API_URL, {
//         method: 'POST',
//         headers: {'Content-Type' : 'application/json'},
//         body: JSON.stringify(user)
//     });
// }
```

- **method: 'POST'**: Define el método HTTP
- **headers**: Debe especificarse manualmente
- **body: JSON.stringify(user)**: Debe convertirse a JSON manualmente
- Más verboso que axios

Comparación Fetch vs Axios:

Aspecto	Fetch	Axios
Parseo JSON	Manual con `.json()`	Automático con `.data`
Headers	Manual	Automático
Body	Manual con `JSON.stringify()`	Automático
Errores HTTP	No lanza error en 404, 500	Lanza error automáticamente
Código	Más verboso	Más conciso

---

9. Componente Principal de React (App.tsx)

Ubicación: `frontend/src/App.tsx`

Código Completo del Archivo:

```
import { useEffect, useState } from "react";
import { createUser, getUsers, type User } from "./services/api";

interface Message {
  name?: string;
  type: "success" | "danger";
```

```
}

//1
interface Errors {
  name?: string;
  email?: string;
}

function App() {
  const [users, setUsers] = useState<User[]>([]);
  const [name, setName] = useState<string>("");
  const [email, setEmail] = useState<string>("");
  const [message, setMessage] = useState<Message | null>(null);
  //2
  const [errors, setErrors] = useState<Errors>({});
  const [loading, setLoading] = useState(false);
  const [isValid, setIsValid] = useState(false);

  const loadUsers = async () => {
    setLoading(true);
    const data = await getUsers();
    setTimeout(() => setLoading(false), 3000);
    setUsers(data);
  };

  //3
  const validateForm = (name: string, email: string): boolean => {
    const newErrors: Errors = {};
    if (!name.trim()) {
      newErrors.name = "El nombre es obligatorio";
    }
    if (!(name.length > 3)) {
      newErrors.name = "El nombre debe tener al menos 3 caracteres";
    }
    if (!email.trim()) {
      newErrors.email = "El email es obligatorio";
    } else if (!email.includes("@")) {
      newErrors.email = "El email no tiene el formato correcto";
    }
    setErrors(newErrors);
    const formOk = Object.keys(newErrors).length === 0;
    setIsValid(formOk);
    return formOk;
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    //4
    if (!validateForm(name, email)) return;

    if (email.toLowerCase().endsWith("example.com")) {
      setErrors((prev) => ({
        ...prev,
        email: "Los correos de example.com no están permitidos.",
```

```
      }));
      setIsValid(false);
      return;
    }
    try {
      await createUser({ name, email });
      setName("");
      setEmail("");
      loadUsers();
      setMessage({ name: "Usuario creado correctamente", type: "success" });
    } catch (error) {
      setMessage({
        name: "No se ha podido guardar el usuario",
        type: "danger",
      });
    }
    setTimeout(() => setMessage(null), 3000);
  };

  useEffect(() => {
    loadUsers();
  }, []);

  //5
  return (
    <div className="container mt-5">
      <div className="card shadow">
        <div className="card-body">
          <h2 className="card-title mb-4 text-center">Usuarios</h2>
          {message && (
            <div className={`alert alert-${message.type} mt-3`}>
              {message.name}
            </div>
          )}

          <form onSubmit={handleSubmit} className="mb-4" noValidate>
            <div className="mb-3">
              <input
                type="text"
                className={`form-control ${errors.name ? "is-invalid" : ""}`}
                value={name}
                placeholder="Nombre Usuario"
                onChange={(e) => {
                  setName(e.target.value);
                  validateForm(e.target.value, email);
                }}
              />
              {errors.name && (
                <div className="invalid-feedback  mb-4">{errors.name}</div>
              )}
            </div>
            <div className="mb-3">
              <input
                type="email"
```

```
                  className={`form-control ${errors.email ? "is-invalid" : ""}`}
                  value={email}
                  placeholder="Email Usuario"
                  onChange={(e) => {
                    setEmail(e.target.value);
                    validateForm(name, e.target.value);
                  }}
                />
                {errors.email && (
                  <div className="invalid-feedback  mb-4">{errors.email}</div>
                )}
              </div>
              <button
                className={`btn btn-${!isValid ? "danger" : "success"}`}
                disabled={!isValid}
              >
                Crear Usuario
              </button>
            </form>
            {loading && <div className="spinner-border" role="status"></div>}
            {!loading && (
              <ul className="list-group">
                {users.map((u) => (
                  <li
                    key={u._id}
                    className="list-group-item d-flex justify-content-between align-
items-center"
                  >
                    {u.name} - {u.email}
                  </li>
                ))}
              </ul>
            )}
          </div>
        </div>
      </div>
    );
  }

  export default App;
```

Explicación Detallada por Secciones:

1. Importaciones

```
import { useEffect, useState } from "react";
import { createUser, getUsers, type User } from "./services/api";
```

- **useEffect**: Hook para efectos secundarios (cargar datos al montar)
- **useState**: Hook para manejar estado local

- **createUser, getUsers**: Funciones del servicio API
- **type User**: Interfaz de usuario

2. Interfaces de TypeScript

```
interface Message {
  name?: string;
  type: "success" | "danger";
}

interface Errors {
  name?: string;
  email?: string;
}
```

- **Message**: Define la estructura de los mensajes de alerta
  - **name**: Texto del mensaje (opcional)
  - **type**: Tipo de alerta ('success' verde o 'danger' rojo)
- **Errors**: Define la estructura de los errores de validación
  - Campos opcionales para name y email

3. Estados del componente

```
const [users, setUsers] = useState<User[]>([]);
const [name, setName] = useState<string>("");
const [email, setEmail] = useState<string>("");
const [message, setMessage] = useState<Message | null>(null);
const [errors, setErrors] = useState<Errors>({});
const [loading, setLoading] = useState(false);
const [isValid, setIsValid] = useState(false);
```

- **users**: Array de usuarios, inicia vacío
- **name**: Valor del input de nombre, inicia vacío
- **email**: Valor del input de email, inicia vacío
- **message**: Mensaje de alerta, inicia null (sin mensaje)
- **errors**: Objeto con errores de validación, inicia vacío
- **loading**: Estado de carga, inicia false
- **isValid**: Indica si el formulario es válido, inicia false

4. Función para cargar usuarios

```
const loadUsers = async () => {
  setLoading(true);
  const data = await getUsers();
  setTimeout(() => setLoading(false), 3000);
```

```
  setUsers(data);
};
```

- **setLoading(true)**: Activa el estado de carga (muestra spinner)
- **await getUsers()**: Llama a la API para obtener usuarios
- **setTimeout**: Espera 3 segundos antes de desactivar el loading (para demostración)
- **setUsers(data)**: Actualiza el estado con los usuarios obtenidos

5. useEffect para carga inicial

```
useEffect(() => {
  loadUsers();
}, []);
```

- Se ejecuta una vez al montar el componente (array de dependencias vacío)
- Carga los usuarios al iniciar la aplicación

6. Función handleSubmit

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!validateForm(name, email)) return;

  if (email.toLowerCase().endsWith("example.com")) {
    setErrors((prev) => ({
      ...prev,
      email: "Los correos de example.com no están permitidos.",
    }));
    setIsValid(false);
    return;
  }
  try {
    await createUser({ name, email });
    setName("");
    setEmail("");
    loadUsers();
    setMessage({ name: "Usuario creado correctamente", type: "success" });
  } catch (error) {
    setMessage({ name: "No se ha podido guardar el usuario", type: "danger" });
  }
  setTimeout(() => setMessage(null), 3000);
};
```

- **e.preventDefault()**: Previene la recarga de página
- **validateForm()**: Valida el formulario antes de enviar
- **Validación adicional**: Rechaza emails de example.com

- **try-catch**: Maneja errores de la petición
- **Limpiar campos**: Vacía name y email después de crear
- **Recargar usuarios**: Actualiza la lista
- **Mensaje temporal**: Muestra mensaje por 3 segundos

---

10. Sistema de Validaciones

El sistema de validaciones es una parte crucial que asegura que los datos ingresados sean correctos antes de enviarlos al servidor.

Paso 1: Definir la interfaz de errores

```
interface Errors {
  name?: string;
  email?: string;
}
```

¿Por qué?

- Define la estructura de los mensajes de error
- Los campos son opcionales (?) porque pueden no tener errores
- TypeScript nos ayudará a evitar errores al manipular estos datos

Paso 2: Crear estados para las validaciones

```
const [errors, setErrors] = useState<Errors>({});
const [isValid, setIsValid] = useState(false);
```

Explicación:

- **errors**: Almacena los mensajes de error para cada campo
- **isValid**: Boolean que indica si el formulario completo es válido
- Ambos estados se actualizan cada vez que se valida el formulario

Paso 3: Crear la función de validación

```
const validateForm = (name: string, email: string): boolean => {
  const newErrors: Errors = {};
  if (!name.trim()) {
    newErrors.name = "El nombre es obligatorio";
  }
  if (!(name.length > 3)) {
    newErrors.name = "El nombre debe tener al menos 3 caracteres";
  }
  if (!email.trim()) {
    newErrors.email = "El email es obligatorio";
```

```
    } else if (!email.includes("@")) {
      newErrors.email = "El email no tiene el formato correcto";
    }
    setErrors(newErrors);
    const formOk = Object.keys(newErrors).length === 0;
    setIsValid(formOk);
    return formOk;
  };
```

Explicación detallada:

a) Objeto temporal de errores

```
const newErrors: Errors = {};
```

- Crea un objeto vacío para acumular errores
- Se llena con mensajes solo si hay problemas

b) Validar nombre no vacío

```
if (!name.trim()) {
  newErrors.name = "El nombre es obligatorio";
}
```

- **name.trim()**: Elimina espacios en blanco al inicio y final
- **!name.trim()**: Si después de trim está vacío, es inválido
- Añade mensaje de error si falla

c) Validar longitud mínima del nombre

```
if (!(name.length > 3)) {
  newErrors.name = "El nombre debe tener al menos 3 caracteres";
}
```

- Verifica que el nombre tenga más de 3 caracteres
- Si ya había un error previo, este lo sobrescribe

d) Validar email no vacío

```
if (!email.trim()) {
  newErrors.email = "El email es obligatorio";
}
```

- Similar a la validación del nombre
- Verifica que no esté vacío

e) Validar formato de email

```
else if(!email.includes('@')){
    newErrors.email = 'El email no tiene el formato correcto'
}
```

- **else if**: Solo se ejecuta si el email NO está vacío
- **includes('@')**: Validación básica de formato de email
- En producción, usarías una regex más compleja

f) Actualizar estado de errores

```
setErrors(newErrors);
```

- Actualiza el estado con todos los errores encontrados
- Esto causa un re-render y muestra los mensajes

g) Determinar si el formulario es válido

```
const formOk = Object.keys(newErrors).length === 0;
setIsValid(formOk);
return formOk;
```

- **Object.keys(newErrors)**: Obtiene un array con las claves del objeto
- **length === 0**: Si no hay claves, no hay errores
- Actualiza **isValid** y retorna el resultado

Paso 4: Integrar validación en los inputs

```
<input
  type="text"
  className={`form-control ${errors.name ? "is-invalid" : ""}`}
  value={name}
  placeholder="Nombre Usuario"
  onChange={(e) => {
    setName(e.target.value);
    validateForm(e.target.value, email);
  }}
/>;
{
```

```
  errors.name && <div className="invalid-feedback  mb-4">{errors.name}</div>;
}
```

Explicación:

a) Clase dinámica para el input

```
className={`form-control ${errors.name ? 'is-invalid' : '' }` }
```

- **form-control**: Clase de Bootstrap para el estilo del input
- **is-invalid**: Clase que se añade si hay error (borde rojo)
- **Template literal**: Permite concatenar clases dinámicamente
- **errors.name ?**: Si existe error, añade 'is-invalid', si no, string vacío

b) Validación en tiempo real

```
onChange={e => {
    setName(e.target.value);
    validateForm(e.target.value,email);
}}
```

- **setName(e.target.value)**: Actualiza el estado con el valor del input
- **validateForm(e.target.value,email)**: Valida inmediatamente
- El usuario ve los errores mientras escribe

c) Mensaje de error

```
{
  errors.name && <div className="invalid-feedback  mb-4">{errors.name}</div>;
}
```

- **errors.name &&**: Renderizado condicional (solo si hay error)
- **invalid-feedback**: Clase de Bootstrap para mensajes de error
- Muestra el mensaje específico del error

Paso 5: Validación antes de enviar

```
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!validateForm(name, email)) return;
  // ... resto del código
};
```

Explicación:

- **validateForm()**: Se llama una última vez antes de enviar
- **if (!validateForm(...)) return**: Si no es válido, sale de la función
- No hace la petición al servidor si hay errores

Paso 6: Validación adicional personalizada

```
if (email.toLowerCase().endsWith("example.com")) {
  setErrors((prev) => ({
    ...prev,
    email: "Los correos de example.com no están permitidos.",
  }));
  setIsValid(false);
  return;
}
```

Explicación:

- Validación de negocio específica
- **toLowerCase()**: Convierte a minúsculas para comparar
- **endsWith()**: Verifica si termina con el dominio
- **setErrors(prev => ({ ...prev, ... }))**: Mantiene errores previos y añade uno nuevo
- **return**: Impide que continúe el envío

Paso 7: Deshabilitar botón si no es válido

```
<button
  className={`btn btn-${!isValid ? "danger" : "success"}`}
  disabled={!isValid}
>
  Crear Usuario
</button>
```

Explicación:

- **disabled={!isValid}**: Botón deshabilitado si formulario inválido
- **btn-danger o btn-success**: Botón rojo si inválido, verde si válido
- Mejora la UX indicando claramente el estado

---

11. Elemento de Carga (Loading)

El elemento de carga proporciona feedback visual al usuario mientras se procesan peticiones asíncronas.

Paso 1: Crear el estado de loading

```
const [loading, setLoading] = useState(false);
```

Explicación:

- **false**: Valor inicial (no está cargando)
- Controla cuándo mostrar el spinner

Paso 2: Activar loading al cargar datos

```
const loadUsers = async () => {
  setLoading(true);
  const data = await getUsers();
  setTimeout(() => setLoading(false), 3000);
  setUsers(data);
};
```

Explicación detallada:

a) Activar el loading

```
setLoading(true);
```

- Se ejecuta ANTES de la petición
- Muestra inmediatamente el spinner

b) Hacer la petición

```
const data = await getUsers();
```

- Petición asíncrona al backend
- Puede tardar varios segundos según la conexión

c) Desactivar el loading con retraso

```
setTimeout(() => setLoading(false), 3000);
```

- **setTimeout**: Espera 3 segundos (3000ms)
- **Propósito**: Asegurar que el usuario vea el spinner
- En producción real, usarías:

```
setLoading(false);
```

directamente después de obtener los datos

d) Actualizar los usuarios

```
setUsers(data);
```

- Guarda los datos en el estado
- Esto causa un re-render y muestra la lista

Paso 3: Renderizar condicionalmente según loading

```
{
  loading && <div className="spinner-border" role="status"></div>;
}
{
  !loading && (
    <ul className="list-group">
      {users.map((u) => (
        <li
          key={u._id}
          className="list-group-item d-flex justify-content-between align-items-
center"
        >
          {u.name} - {u.email}
        </li>
      ))}
    </ul>
  );
}
```

Explicación detallada:

a) Mostrar spinner si está cargando

```
{
  loading && <div className="spinner-border" role="status"></div>;
}
```

- **loading &&**: Renderizado condicional (solo si loading es true)
- **spinner-border**: Clase de Bootstrap que crea una animación circular
- **role="status"**: Atributo de accesibilidad
- Se muestra mientras loading = true

b) Mostrar lista si NO está cargando

```
{
  !loading && (
    <ul className="list-group">
      {users.map((u) => (
        <li
          key={u._id}
          className="list-group-item d-flex justify-content-between align-items-
center"
        >
          {u.name} - {u.email}
        </li>
      ))}
    </ul>
  );
}
```

- **!loading &&**: Solo si loading es false
- **users.map()**: Itera sobre cada usuario
- **key={u._id}**: Key única requerida por React para listas
- **list-group**: Clases de Bootstrap para lista estilizada

Flujo completo del loading:

1. **Usuario abre la app**
   - useEffect se ejecuta
   - loadUsers() es llamado
2. **setLoading(true)**
   - Estado cambia a true
   - Se muestra el spinner
   - La lista desaparece
3. **Petición al backend**
   - getUsers() hace fetch
   - Puede tardar 100ms - varios segundos
4. **Datos recibidos**
   - setTimeout espera 3 segundos adicionales
   - Esto asegura que el usuario vea el spinner
5. **setLoading(false)**
   - Estado cambia a false
   - Spinner desaparece

- Lista de usuarios aparece

Mejoras opcionales para producción:

```
const loadUsers = async () => {
  setLoading(true);
  try {
    const data = await getUsers();
    setUsers(data);
  } catch (error) {
    console.error("Error cargando usuarios:", error);
    // Mostrar mensaje de error
  } finally {
    setLoading(false); // Siempre se ejecuta
  }
};
```

Ventajas:

- **try-catch**: Maneja errores de red
- **finally**: Se ejecuta siempre, incluso si hay error
- **No hay setTimeout**: Desactiva loading inmediatamente

---

12. Sistema de Alertas de Mensajes

El sistema de alertas proporciona feedback al usuario sobre operaciones exitosas o fallidas.

Paso 1: Definir la interfaz de mensaje

```
interface Message {
  name?: string;
  type: "success" | "danger";
}
```

Explicación:

- **name**: Texto del mensaje (opcional)
- **type**: 'success' para éxito (verde) o 'danger' para error (rojo)
- **Union type**: TypeScript solo permite estos dos valores

Paso 2: Crear el estado del mensaje

```
const [message, setMessage] = useState<Message | null>(null);
```

Explicación:

- **Message | null**: Puede ser un objeto Message o null (sin mensaje)
- **null**: Estado inicial (sin alerta visible)

Paso 3: Renderizar la alerta condicionalmente

```
{
  message && (
    <div className={`alert alert-${message.type} mt-3`}>{message.name}</div>
  );
}
```

Explicación detallada:

a) Renderizado condicional

```
{message && ...}
```

- Solo renderiza si message no es null
- Cuando message = null, no se muestra nada

b) Clases dinámicas de Bootstrap

```
className={`alert alert-${message.type} mt-3`}
```

- **alert**: Clase base de Bootstrap para alertas
- **alert-${message.type}**: Interpolación dinámica
  - Si type = 'success': 'alert-success' (fondo verde)
  - Si type = 'danger': 'alert-danger' (fondo rojo)
- **mt-3**: Margen superior (Bootstrap spacing)

c) Contenido del mensaje

```
{
  message.name;
}
```

- Muestra el texto del mensaje
- Ejemplo: "Usuario creado correctamente"

Paso 4: Mostrar mensaje de éxito

```
try {
  await createUser({ name, email });
  setName("");
  setEmail("");
  loadUsers();
  setMessage({ name: "Usuario creado correctamente", type: "success" });
} catch (error) {
  setMessage({ name: "No se ha podido guardar el usuario", type: "danger" });
}
setTimeout(() => setMessage(null), 3000);
```

Explicación detallada:

a) Bloque try - Operación exitosa

```
try{
    await createUser({name, email });
    setName('');
    setEmail('');
    loadUsers();
    setMessage({name:'Usuario creado correctamente', type:'success'});
}
```

- **try**: Intenta ejecutar el código
- **await createUser()**: Crea el usuario en el backend
- **Limpiar campos**: Vacía los inputs
- **loadUsers()**: Recarga la lista actualizada
- **setMessage(...)**: Muestra alerta verde de éxito

b) Bloque catch - Error en la operación

```
catch (error) {
    setMessage({name:'No se ha podido guardar el usuario', type:'danger'});
}
```

- **catch**: Se ejecuta si hay algún error
- Errores posibles:
  - Backend no disponible
  - Error de red
  - Error de validación del backend
- **setMessage(...)**: Muestra alerta roja de error

c) Ocultar mensaje automáticamente

```
setTimeout(() => setMessage(null), 3000);
```

- **setTimeout**: Programa la ejecución después de un delay
- **3000**: 3 segundos (3000 milisegundos)
- **setMessage(null)**: Oculta la alerta después de 3 segundos
- Se ejecuta tanto si fue éxito como error

Flujo completo del sistema de alertas:

1. **Usuario envía formulario**
   - handleSubmit se ejecuta
   - Validaciones pasan
2. **Petición al backend**
   - createUser() hace POST
   - Backend procesa la petición
3. **Respuesta exitosa**
   - Backend responde 201 Created
   - Se ejecuta el bloque try
   - setMessage muestra alerta verde
   - Texto: "Usuario creado correctamente"
4. **O respuesta con error**
   - Backend no responde o error 500
   - Se ejecuta el bloque catch
   - setMessage muestra alerta roja
   - Texto: "No se ha podido guardar el usuario"
5. **Mensaje visible**
   - Alerta aparece en la UI
   - Permanece visible durante 3 segundos
6. **Ocultando mensaje**
   - setTimeout se ejecuta
   - setMessage(null) oculta la alerta
   - La alerta desaparece suavemente

Ejemplo visual de las alertas:

Alerta de éxito:

```
<div class="alert alert-success mt-3">Usuario creado correctamente</div>
```

- Fondo verde claro
- Texto verde oscuro
- Icono de éxito (opcional)

Alerta de error:

```
<div class="alert alert-danger mt-3">No se ha podido guardar el usuario</div>
```

- Fondo rojo claro
- Texto rojo oscuro
- Icono de error (opcional)

Mejoras opcionales:

a) Alerta con botón de cierre

```
{
  message && (
    <div
      className={`alert alert-${message.type} alert-dismissible fade show mt-3`}
    >
      {message.name}
      <button
        type="button"
        className="btn-close"
        onClick={() => setMessage(null)}
      ></button>
    </div>
  );
}
```

b) Múltiples mensajes en cola

```
const [messages, setMessages] = useState<Message[]>([]);

const addMessage = (msg: Message) => {
  const id = Date.now();
  setMessages((prev) => [...prev, { ...msg, id }]);
  setTimeout(() => {
    setMessages((prev) => prev.filter((m) => m.id !== id));
  }, 3000);
};
```

c) Iconos en las alertas

```
{
  message && (
    <div className={`alert alert-${message.type} mt-3`}>
      {message.type === "success" ? "✅" : "❌"} {message.name}
    </div>
  );
}
```

🚀 Ejecución del Proyecto

1. Asegúrate de tener MongoDB corriendo

```
# Windows - Abre MongoDB Compass o verifica que el servicio esté activo
# O ejecuta en terminal:
mongod
```

2. Ejecutar el Backend

```
# Desde la raíz del proyecto
npm run dev
```

Deberías ver:

```
Connected to MongoDB
Server is running on port 4000
```

3. Ejecutar el Frontend

```
# Abre otra terminal
cd frontend
npm run dev
```

Deberías ver:

```
VITE v5.x.x  ready in xxx ms

➜  Local:   http://localhost:5173/
➜  Network: use --host to expose
```

4. Abrir en el navegador

Navega a: http://localhost:5173/

🎯 Flujo Completo de la Aplicación

Flujo al abrir la aplicación:

1. **React se monta** → useEffect ejecuta loadUsers()
2. **loadUsers()** → setLoading(true), muestra spinner
3. **Petición GET** → Frontend llama a http://localhost:4000/api/users
4. **Backend responde** → Mongoose encuentra usuarios en MongoDB
5. **Datos llegan** → Frontend actualiza users, oculta spinner, muestra lista

Flujo al crear un usuario:

1. **Usuario escribe nombre y email** → validateForm() valida en tiempo real
2. **Usuario hace clic en "Crear Usuario"** → handleSubmit se ejecuta
3. **Validación final** → validateForm() confirma que todo es válido
4. **Petición POST** → Frontend envía datos a http://localhost:4000/api/users
5. **Backend procesa** → Express crea nuevo documento en MongoDB
6. **Éxito** → Backend responde 201, Frontend muestra alerta verde
7. **Actualización** → Frontend recarga usuarios con loadUsers()
8. **Mensaje desaparece** → Después de 3 segundos

📝 Resumen de Conceptos Clave

Backend:

- **Express**: Framework web para Node.js
- **Mongoose**: ODM para MongoDB con esquemas tipados
- **CORS**: Permite peticiones desde dominios diferentes
- **TypeScript**: Añade tipos estáticos a JavaScript
- **Separación de responsabilidades**: Models, Routes, App, Server

Frontend:

- **React**: Librería para interfaces de usuario
- **Hooks**: useState (estado), useEffect (efectos secundarios)
- **Axios**: Cliente HTTP más simple que fetch
- **Bootstrap**: Framework CSS para estilos
- **TypeScript**: Tipos para prevenir errores

Validaciones:

- **Validación en tiempo real**: Feedback inmediato al usuario
- **Validación antes de enviar**: Previene peticiones inválidas
- **Múltiples niveles**: Cliente y servidor

- **UX mejorada**: Botón deshabilitado, mensajes claros

Loading:

- **Feedback visual**: Usuario sabe que algo está procesándose
- **Estados de carga**: Controlan qué mostrar
- **Spinner de Bootstrap**: Animación lista para usar

Alertas:

- **Try-catch**: Manejo de errores
- **Feedback contextual**: Éxito verde, error rojo
- **Auto-desaparición**: No molesta al usuario
- **Bootstrap alerts**: Componentes listos

🎓 Conclusión

Esta aplicación fullstack demuestra:

- Comunicación cliente-servidor con REST API
- Validación robusta de formularios
- Feedback visual al usuario (loading, alertas)
- Manejo de errores adecuado
- Código limpio y bien estructurado
- TypeScript para seguridad de tipos
- Buenas prácticas de React y Express

¡Ahora tienes una base sólida para crear aplicaciones fullstack profesionales! 🚀